Programming Assignment 2
Distributed Key-Value Store

Author: ShangJhe Li, Dickson Tio

> **Important Deadlines & Resources**
>
> - **Soft Deadline:** 11/5 (Thursday) 23:59
> - **Hard Deadline:** 11/12 (Thursday) 23:59
> - **Template Repository:** https://github.com/mtmatt/sp-hw2-student-template

Contents

1 Prerequisites 1
2 Overview 2
3 Operations 6
4 Implementation 10
5 General Rules 14
6 Grading 15
7 Submission 17
8 FAQ 18
Appendix 20

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

1 Prerequisites

1.1 Message from TA

Please start this homework ASAP, or at least read the spec thoroughly so that you know how the implementation should be done. And even then, you are not advised to procrastinate at all. The sooner you ask for help, the more we can provide.

Read the full specification carefully before implementing the SP, especially the sections describing process creation, file-descriptor manipulation, pipes, and FIFOs.

As the deadline approaches, please do not give up. Come to TA hours for assistance. If necessary, contact the TAs by email after you have first tried to understand and debug the problem yourself.

- **Class 1:** `swnosp@csie.ntu.edu.tw`
- **Class 2:** `ntusp2026@gmail.com`

The TA Hours of the TAs responsible for this homework are:

- 張迪善：Thursday 14:30 - 15:30 PM, CSIE Building Basement (The Pole in the middle)
- 李尚哲：Wednesday 15:00 - 16:00 PM, CSIE Building Basement (The Pole in the middle)

2 Overview

In this homework, you will build a **student program** (SP) that becomes a tree of processes. Every process in the tree is a **node**, and every node has its own key-value table. The judge starts the root node, **SP main**, then sends commands to grow the tree, store and delete key-value pairs, and look up keys. This is a continuously asynchronous process, and the judge can send each command to any node it wants.

On look-ups, if a node does not have the key, it will ask its parent. If the parent does not have the key either, it asks its own parent, and so on, up to **SP main**. If **SP main** does not have it either, it asks a program provided by the TAs, the **consultant**. (The consultant always knows the answer, but sometimes it fails to answer, more on that later.)

The final result can be viewed as a tree-structured, distributed database with a single root of truth, each process provides its standalone service to the judge, but may query another higher status service if needed..

We design this homework for you to practice using processes and implementing IPC. You will create processes to have them listen to many channels (judge, parent process, child processes) without blocking, and remove subtrees of these processes without leaving any zombies or FIFOs behind. Additionally, you must design a pipe protocol on your own to implement communication between each process, including but not limited to how and what to pass/parse in each message. Since every process maintains multiple communication channels, you are expected to understand and implement polling mechanisms.

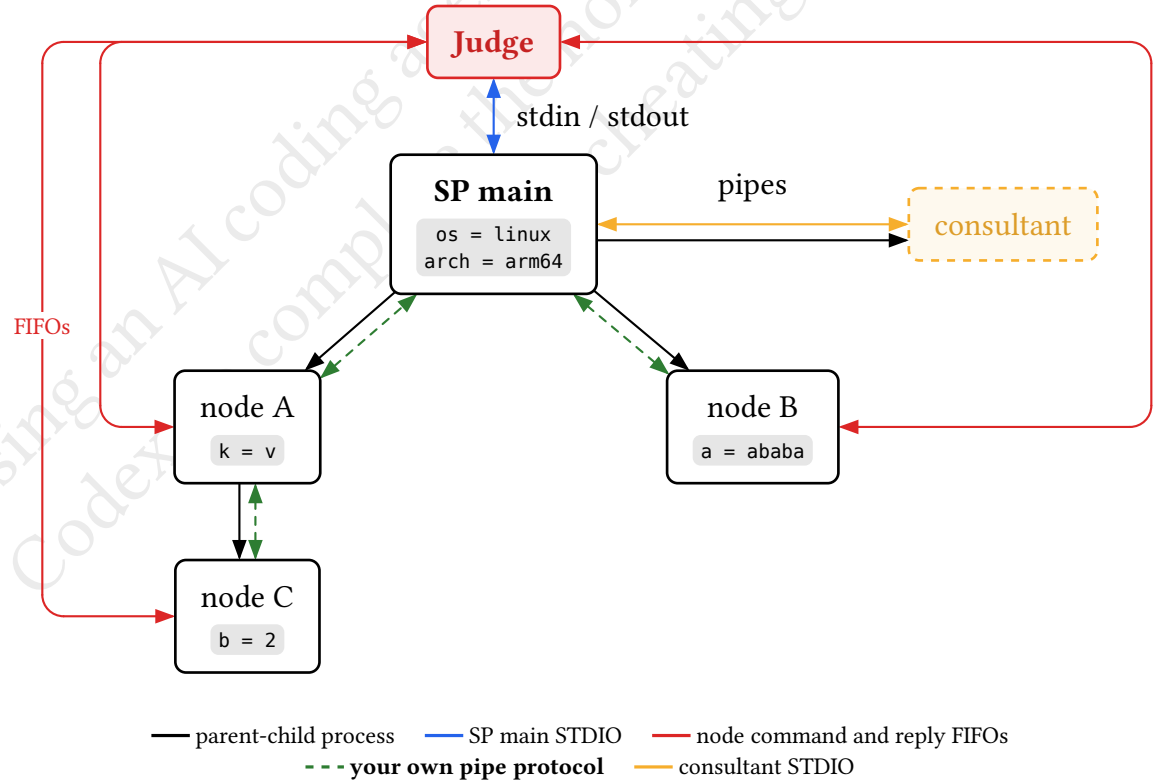


Figure 1: Tree structure example

Keep in mind that the judge talks to **every** node directly, in which **SP main** talks to the judge with its STDIO, while other processes talk to the judge with a pair of FIFOs.

2.1 Terms

2.1.1 Explanation

- **Node**: a process in the tree running the `sp` executable.
- **SP main**: the root node, executed by the judge.
- **Command channel**: where a node reads its commands. STDIN for **SP main**, command FIFO for every other node.
- **Reply channel**: where a node writes its `fetch` answers. STDOUT for **SP main**, and the reply FIFO for every other node.
- **Local table**: all key-value pairs stored by `store` commands sent to a node.
- **Ancestors**: All other nodes along the unique path from a node to the root (**SP main**).

2.1.2 Mapping to Figure 1

Term	In Figure 1
Node	Any process box: SP main, node A, node B, node C.
SP main	Top box, the only node wired to the judge by STDIO.
Command channel	STDIN for SP main; the judge-to-node FIFO for A, B and C.
Reply channel	STDOUT SP main; the node-to-judge FIFO for A, B and C.
Local table	The key-value pairs listed inside each box.
Ancestors	Nodes on the tree-edge path up to SP main (C: A, SP main; A and B: SP main).

2.2 What you build

2.2.1 Initialization

By executing `$ make`, your `Makefile` should compile an executable named `sp` in the submission directory. The judge creates **SP main** by executing `$ ./sp` in that directory without any command-line arguments.

2.2.2 Constructing the tree

Given every node in the tree is your `sp` executable, you must create new processes by using `fork()`, then `exec()` your `sp` as the child process, such that every node starts with a new address space (no inherited memory from its parent).

Note that:

- If you `exec()` it as `./sp`, please do not change directory in any node.
- How a new node knows its role and its FIFO channel paths is up to you (command-line arguments, environment variables, or pipes all work).

2.2.3 Environment variable

The judge sets one environment variable for **SP main**, and reads it at runtime. Please do not hard-code a path from your own machine (it won't exist when we grade).

Variable	Purpose
`JUDGE_CONSULTANT`	Absolute path of the consultant **SP main** starts out with.

Using AI coding agents (e.g. Claude Code or Codex, etc.) to complete the homework for you counts as cheating.

2.2.4 Command Processing

Every node must:

- Treat each line ending with the newline byte `'\n'` on its command channel as one command.
- Handle the commands in the order they arrive.
- Write only `fetch` answers to its reply channel, one line each, each sent immediately (**do not** buffer multiple replies to be sent as one).
- Run indefinitely until a `kill` command removes it. If **SP main** reads end-of-file on standard input, it should act as if it received `kill`.

> ⚠ **STDOUT is a protocol channel**
>
> **DO NOT** print debug messages to STDOUT of **SP main**. Additionally, without redirection, other nodes you create inherit these prints, hence anything the tree nodes print will corrupt replies from **SP main**. If you need debug messages for any node, print to **STDERR** instead.

2.3 Keys and values

2.3.1 K-V format

Keys and values are UTF-8 text without ASCII control characters, we guarantee there will be no newline, carriage return, tab, or NUL characters.

- A **key** contains no space, and is never empty.
- A **value** is everything after the single space that follows the **key**, may contain spaces (even at the start or the end), and is never empty.
  - E.g. upon receiving the `store` operation with argument `sphw2  is fun buthardd  \n`
  - The **key** is `sphw2`
  - The **value** is `  is fun buthardd  `
  - Note that for the example value there are space characters, **one** at the start and **three** at the end.

2.3.2 K-V memory limits

At any given time, if you correctly add and remove key-value pairs, the local table held by any node will add up to **less than a quarter** of the test's soft memory limit for each process. This limit accounts for its local table, the lookups currently being handled or is passing on, and for **SP main**, its consultant runs. This guarantees room for three more copies while you parse and forward a key-value pair.

2.3.3 K-V test cases

Note that in grading test cases:

- Groups 1 to 3, every key and value is shorter than 1024 bytes.
- Group 4 has no definite max length besides the memory limit, hence you **should not** assume a fixed-size buffer will always work.

Using an AI coding agent, such as Claude Code, to complete the homework for you counts as cheating.

2.4 Limitations

With correct implementation and judge command execution, we guarantee that at any given time, scenarios generated by our test cases will have no more than 32 nodes (**SP main** included) in the tree, and the tree depth is no more than 8 levels deep, with **SP main** as level 1.

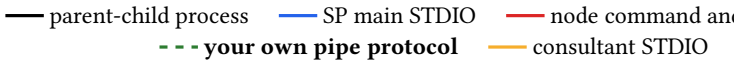
Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

3 Operations

The judge sends six kinds of commands. In this section, we follow each of them end to end on a small tree, so you can see every message it causes and which process sends it. The numbers on the arrows indicate the order of the steps.

Note that this section only serves as a guide for better understanding, please refer to Implementation (Section 4) for rules and specifications.

3.1 Graph line definitions



3.2 `create`: add a child process

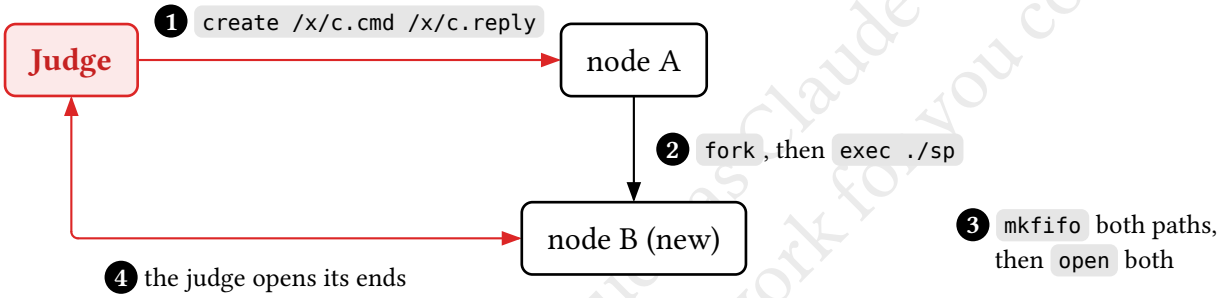


Figure 2: Node A receives `create` and starts node B.

1. The judge writes `create <command-fifo> <reply-fifo>` to node A.
2. Node A calls `fork()`, and the child process calls `exec()` on your `sp` to create node B as a direct child process of node A.
3. Node B creates both FIFOs with `mkfifo()`, then opens the command FIFO for reading and the reply FIFO for writing.
4. The judge opens the other end of each FIFO, such that the `open()` calls of node B can return. From this point on, the judge sends commands to node B directly.

Note that if node A is **SP main**, the judge sends `create` through STDIN instead of a command FIFO, but Node B still uses FIFOs.

3.3 `store` and `delete`: change a local table

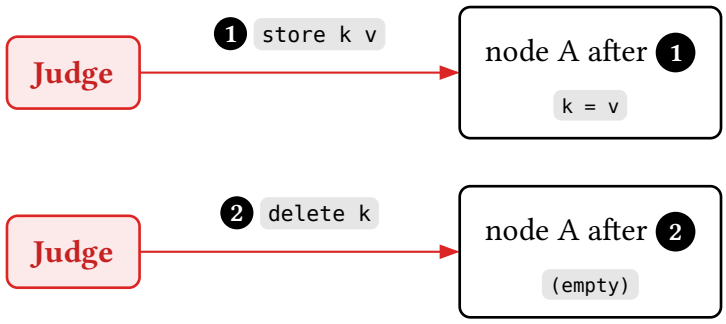


Figure 3: `store` and `delete` change only the table of the receiving node.

1. `store k v` puts the pair into the local table of node A. No other node is affected.
2. `delete k` removes `k` from the local table of node A.

Note that neither command gets a reply, so the judge will not wait for either of them.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

3.4 `fetch` case 1: key is in the receiving node

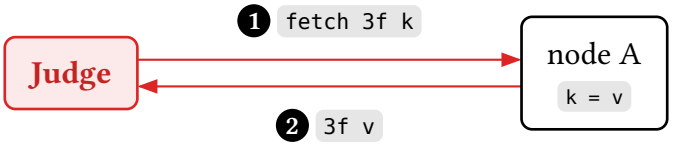


Figure 4: Node A answers from its own table.

1. The judge writes `fetch 3f k` to the command channel of node A. Here, `3f` is the hash the judge chose for this request.
2. Node A holds `k` in its local table, so it writes `3f v` to its reply channel.

3.5 `fetch` case 2.1: the key is in the parent

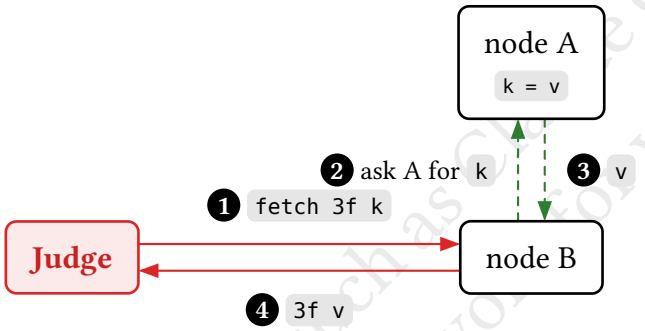


Figure 5: Node B misses and asks its parent, which holds the key.

1. The judge writes `fetch 3f k` to node B.
2. Node B does not hold `k`, so it asks its parent, node A, using the protocol you designed.
3. Node A holds `k`, and sends `v` back down to node B.
4. Node B writes `3f v` to its reply channel.

Note that if node A misses as well, it will ask its parent again.

3.6 `fetch` case 2.2: the key is in an ancestor

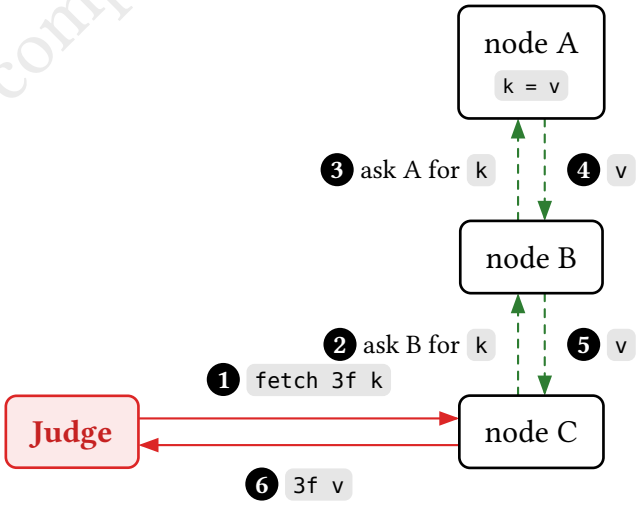


Figure 6: node C misses and asks its parent, propagating to node A, which holds the key.

1. The judge writes `fetch 3f k` to node C.
2. Node C does not hold `k`, so it asks its parent, node B, using the protocol you designed.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

3. Node B does not hold `k` either, so it asks its parent, node A, using the same protocol.
4. Node A holds `k`, and sends `v` back down to node B.
5. Node B forwards `v` down to node C.
6. Node C writes `3f v` to its reply channel.

Note that `fetch` case 2.1 is a special case of `fetch` case 2.2 where the key is found at the first parent queried.

3.7 `fetch` case 3: no node holds the key

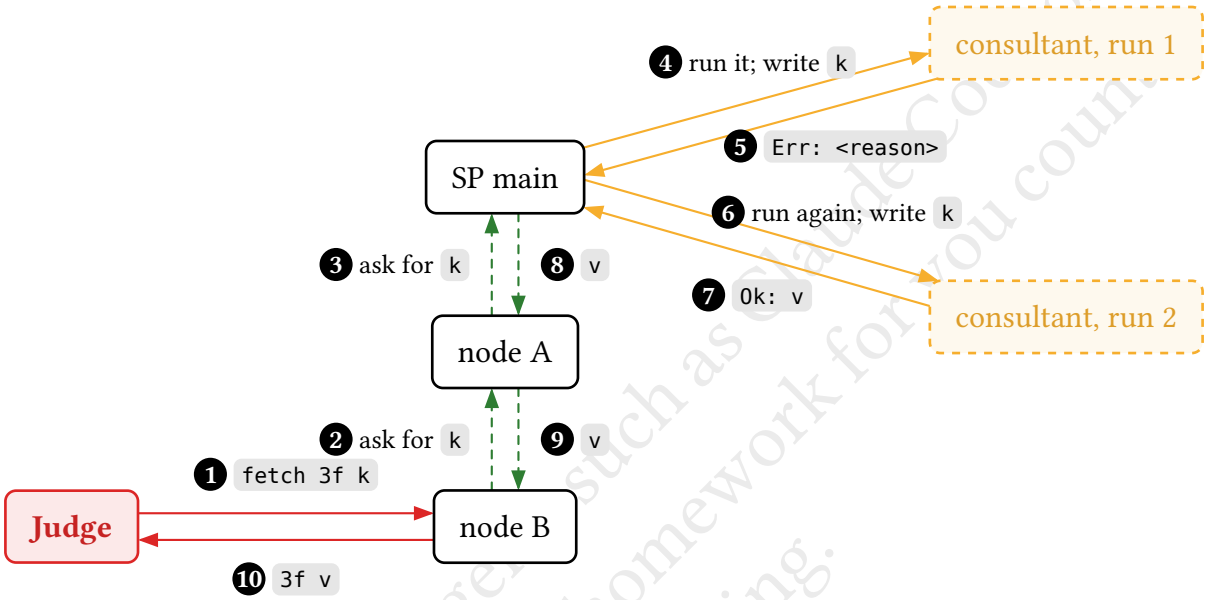


Figure 7: Every node up to root misses, **SP main** runs the consultant until success

1. The judge writes `fetch 3f k` to node B.
2. Node B misses, and asks node A.
3. Node A misses, and asks **SP main**.
4. **SP main** misses as well, so it calls `fork()` and `exec()` on the consultant, then writes `k` followed by a newline to the STDIN of the consultant.
5. This run fails, either by answering `Err: <reason>` or by exiting before a complete line is written. The consultant terminates and **SP main** reaps the process.
6. **SP main** starts a new run of the current consultant for the same key.
7. The new run answers `Ok: v` to its STDOUT and terminates, **SP main** reaps the process.
8. **SP main** sends `v` down to node A,
9. node A sends `v` down to node B,
10. node B writes `3f v` to its reply channel.

Note that if the first run answers `Ok: v`, steps 5 and 6 do not happen, and the first consultant run is the one answering in step 7. If a new run fails as well, **SP main** loops step 5 and 6 until one of them answers `Ok: <value>`.

Using an AI coding agent such as Claude Code or Codex to complete the homework for you counts as cheating.

3.8 `kill`: remove a subtree

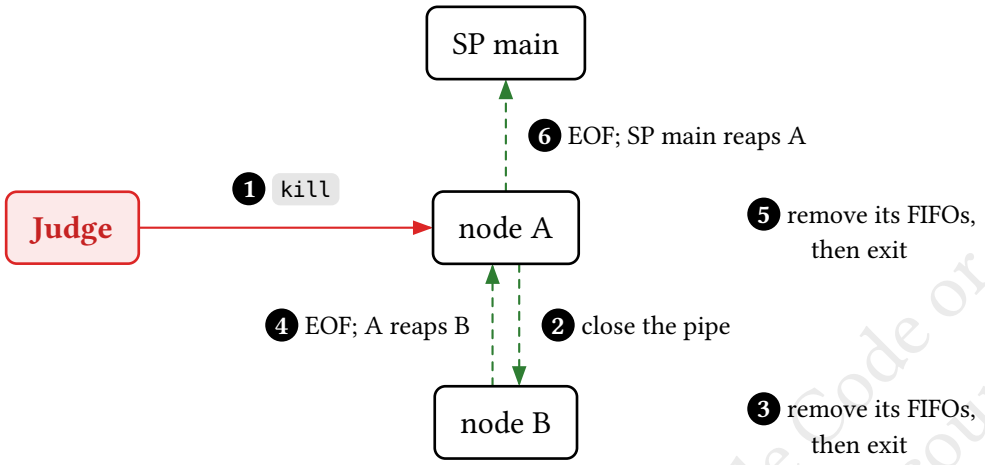


Figure 8: `kill` sent to node A removes node A and node B.

1. The judge writes `kill` to node A.
2. Node A tells its child to exit, for example by closing its end of the pipe to node B (or by sending a message from your protocol).
3. Node B removes the FIFOs it created, then exits.
4. Node A sees EOF on the pipe of node B (or receives a message from your protocol), and reaps node B with `waitpid()`.
5. After all of its children are reaped, node A removes its own FIFOs, then exits.
6. **SP main** sees EOF on the pipe of node A, (or receives a message from your protocol), reaps node A and cleans the file descriptors used for node A.

Note that a `kill` sent to **SP main** removes the whole tree in the same manner, after which **SP main** exits with status 0 and is reaped by the judge.

3.9 `update`: switch consultants

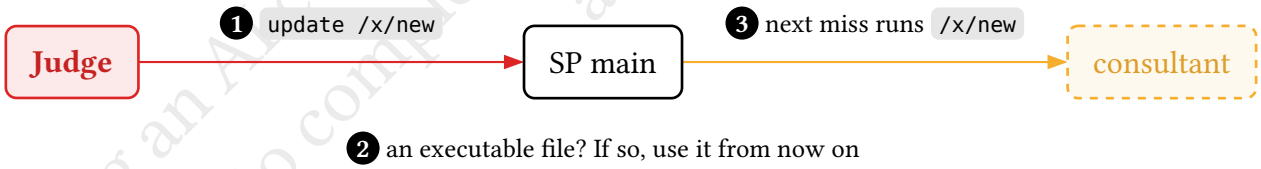


Figure 9: `update` changes the consultant for later runs.

1. The judge writes `update /x/new` to **SP main**.
2. If `/x/new` is a regular file that **SP main** can execute, it uses this file for every consultant run from now on. Otherwise, the command is ignored.
3. The next `fetch` miss that reaches **SP main** runs `/x/new`.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating

4 Implementation

4.1 `create`

4.1.1 Requirement

```
create <command-fifo> <reply-fifo>                                    Protocol
```

Add a new child process under the node that received the command. Both FIFO paths are absolute and contain no whitespace.

1. Call `fork()`, and in the child process call `exec()` on your `sp`. The new node must be a direct child of the receiving node, and each `create` starts **exactly one** new process.
2. The new node creates both FIFOs with `mkfifo()` before opening either of them. The directory of the paths is guaranteed to exist, while the paths themselves do not.
3. The new node opens the command FIFO for reading and the reply FIFO for writing, in either order. Each blocking `open()` returns once the judge has opened the other end, which only happens after both FIFOs exist.
4. Its local table starts empty.

The judge opens both FIFOs before sending any other command to any node, and keeps its ends open until the node is removed. You may remove the FIFO paths as soon as the new node has opened both of them. Otherwise, remove them when the node is killed.

4.1.2 Knowledge check

1. For parent-child pipe communication to work, what must be done?
   - `pipe()` is the obvious answer, but how much do you need?
2. After you use `pipe()`, do you need to manage the file descriptors?
   - you have to close file descriptor after reaping a child process, can you make this easier?

> **Tip**
>
> 1. An `open()` with `O_NONBLOCK` may fail if it is called before the judge has opened its end.
> 2. Open the FIFOs with `O_CLOEXEC`, or set `FD_CLOEXEC` on them right after opening, such that the nodes you `create` later do not inherit them.

4.2 `store` and `delete`

4.2.1 Requirement

```
store <key> <value>                                                   Protocol
```

`store` puts the key-value pair in the receiving node's local table.

- If the **key** already exists, any value already stored under `<key>` is replaced with the new one.

```
delete <key>                                                          Protocol
```

`delete` removes `<key>` from the receiving node's local table.

- If the key is not there, it does nothing.

- The same key in other nodes are unaffected.

This command requires no response to the judge.

4.2.2 Knowledge check

Given group 4 test cases do not limit the key-value length:

- Is it possible that single `read()` may not completely receive the entire string?
- What happens when your local table could not store the key-value pair?

> **Tip**
>
> If a fixed-size buffer is not guaranteed to work, consider the following:
>
> 1. An intermediate buffer to store the bytes being read from the command channel.
> 2. A dynamically allocated key-value storage structure.

4.3 `fetch`

4.3.1 Requirement

```
fetch <hash> <key>                                                    Protocol
```

`<hash>` is a 1 to 16 digit lowercase hexadecimal value (equivalent to a 64-bit integer) that the judge chose for this request. No two concurrent `fetch` commands share the same hash.

Reply with the value that this lookup finds:

1. Look up the receiving node's local table.
2. On miss ask the parent, which will then look up its own local table and asks its own parent, and so on, up to **SP main**. The nearest ancestor that holds the key is the value used for this `fetch`.
3. If **SP main** does not hold the key either, it runs the consultant (Section 4.3.3) and queries it for an answer.
4. The answered value travels back down to the node that received the `fetch` command, which then writes this line to its reply channel:

   ```
   <hash> <value>                                                     Protocol
   ```

Every `fetch` has a value, there are no cases where the value is not found, or where the reply is anything other than stated in above protocol. The reply must arrive within the test's time limit (Section 5.2), which a lookup that deadlocks will most certainly exceed.

> **Do not cache**
>
> Every miss walks the ancestor path, and every miss at **SP main** runs the consultant again. A consultant's answer for the same key may change from one run to the next, even from the same path, so a cached answer will eventually be wrong.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

4.3.2 Many lookups at once

A node may receive several `fetch` commands before it has answered the first, and other nodes may be looking keys up at the same time through the same ancestor path. A node may answer its pending `fetch` commands in any order, as the judge matches each reply to its request by the hash.

> **Tip**
>
> Several lookups can be in process between a parent-child at once, some going up and some going down, give each lookup state an identifier in your own protocol, such that you know the current state of every request and where they need to go next.

4.3.3 The consultant

SP main starts with the consultant named by `JUDGE_CONSULTANT` and switches only on an accepted `update`, refer to (Section 4.5) for details.

To ask the consultant to look up the `<value>` of a `<key>`:

1. `fork()`, then `exec()` the current consultant path with no arguments, and its STDIO connected to pipes.
2. Write `<key>` followed by a newline character to its STDIN, in the format:

   ```
   <key>
   ```
   Protocol

   - It is guaranteed that every consultant reads the whole key line before it answers or exits, and none of them hangs.
3. Read one line from the consultant STDOUT, reap the consultant process, close the corresponding file descriptors. The reply from the consultant is in format:
   1. ```
      Err: <reason>
      ```
      Protocol

      - Upon receiving `Err` or an exit before a complete line means this run failed. **SP main** must start a new run of the current consultant for the same key. Repeat this process until receiving the `Ok` response.
      - Some consultants fail on purpose, but eventually a later run is guaranteed to succeed, the exit status before that does not matter.
   2. ```
      Ok: <value>
      ```
      Protocol

      - Everything after `Ok:` is the `<value>`, which follows the same rules as a local table stored value.

Note that only **SP main** runs the consultant, hence every consultant is a child process of **SP main**, and you may run several at once when several misses reach **SP main** together.

4.3.4 Knowledge check

It is easy to pass a `fetch` request up to root since every child only has one parent process, but what about sending the `<value>` back down?

1. How would a process know which child to send the received `<value>` to?

Using an AI coding agent, such as Claude Code or Codex, to complete this homework for you counts as cheating.

2. Are there any information you can use identify which node in the tree is the `fetch` request coming from?
3. If you know which node to send the `<value>`, can you find the unique traversal path?
4. When should the identification start in your protocol?

4.4 `kill`

4.4.1 Requirement

`kill` `Protocol`

Remove the receiving node and all of its descendants.

When the command is executed:

- Every process in the subtree has returned.
- Each process is reaped by its own parent, the receiving node by its parent, and **SP main** by the judge.
- A parent reaps a child as soon as it notices the exit, without waiting for another command, and must not exit before it has reaped all of its children, such that no zombie processes are ever created.
- Every FIFO created in/for the subtree has been removed.
- If **SP main** is the receiving node, none of its consultant runs is still running or unreaped.

A `kill` sent to **SP main** ends the test, and **SP main** must exit with status `0` only after the entire process tree is terminated..

> **Tip**
>
> - To tell a child to exit, you may close your write end of its pipe such that the child process sees EOF.
> - To learn that a child has exited, you may try to detect EOF on its pipe, then reap it with `waitpid()`.

> **A stray descriptor hangs `kill`**
>
> A child that inherited another pipe's write end keeps that pipe's reader from ever seeing EOF, so the tree termination never finishes. Remember to close the unused pipe ends right after `fork()`, and set `FD_CLOEXEC` on file descriptors that new processes must not inherit.

4.4.2 Knowledge check

Given a parent process reaps its children, and all child processes must be reaped before the parent terminates, what is the order of execution when receiving the `kill` command?

- In your protocol, how do you relay this message to all your descendants?
- How do you make sure the leaf nodes of the subtree are terminated and reaped before any parent process?

4.5 `update`

4.5.1 Requirement

```
update <consultant-path>                                    Protocol
```

The judge sends this only to SP main. `<consultant-path>` is absolute and contains no whitespace.

- If `<consultant-path>` names a regular file that you can execute, use it for every consultant run from now on.
  - Otherwise, ignore the command and keep the current consultant.
- Once you accept a path, the judge keeps the executable valid until the next `update`, so checking it once when the command arrives is enough.
- A consultant that is already running finishes normally.

4.5.2 Knowledge check

Given the `<consultant-path>` has to be a **regular file** with **execute permission**, how do you check these attributes during runtime?

5 General Rules

5.1 Ordering

Commands sent to the same node take effect in the order the judge wrote them. Commands sent to different nodes have no order between them, a node can be handling one command while its parent is still on an older one.

The judge never asks for a result that depends on how commands on different nodes interleave. Given every node behaves correctly under any interleaving that remains, below race condition preventions by the judge are guaranteed:

- After it sends `store k v` to node A, it does not send a `fetch` for `k` to a descendant of A until a reply to a later `fetch` sent to A shows that A has handled the `store`.
- It does not `kill` a subtree while a `fetch` it sent inside that subtree is unanswered.
- After a `kill`, it sends no other command until the `kill` has taken effect, and only then may it reuse the removed nodes' FIFO paths.
- It does not send a valid `update` to swap out the consultant while a `fetch` that requires the previous consultant is unanswered.

5.2 Time limits

Each test sets a time limit, which is 5 seconds in the public tests, within this 5 second limit:

- All new nodes must be created and made to open both of their FIFOs
- Every `fetch` command must be answered.
- All `kill` commands must take effect, taking longer leaves its processes behind as leftovers.

5.3 Restrictions

- A key-value pair lives only in the local table of the node that received its `store`.
- A `<value>` may travel between nodes to answer a `fetch`, but no other process may keep it afterward, whether in memory, a file, or shared memory.

Using AI coding agent, such as Claude Code, or Codex, to complete the homework for you counts as cheating.

- **DO NOT** use `system()` or `popen()`. You should only start one and only one process to perform a `create` command, or to run a consultant.

6 Grading

6.1 Score distribution

The judge scores your submission out of 100 points, divided into four progressive groups. Each group may use every command from the groups before it, and every test ends by removing the whole tree. Your final score for this homework is normalized to 7 points.

Group	What it demonstrates	Points	Public	Private	Of 7
1	`store`, `delete`, `fetch`, and `kill` on SP main alone, with no misses.	7	2.1	4.9	0.49
2	`create` and lookups through ancestors, with several lookups in progress at the same time.	21	6.3	14.7	1.47
3	Misses that reach the consultant, and `update`.	29	8.7	20.3	2.03
4	`kill` below SP main, long keys and values, and resource hygiene.	43	12.9	30.1	3.01
	Total	**100**	**30**	**70**	**7**

Table 1: Points per group, out of 100, and what they are worth out of 7.

Within every group, public tests contribute 30% of the points, and private tests contribute the remaining 70%. The public tests come with the template, so you can run them yourself (Section 6.4), while the private tests are only run when we grade. The points of each part are split equally among its tests. E.g., the public part of group 4 is worth 12.9 points across its 5 tests, which is 2.58 points each.

Note that passing every public test does not guarantee that you will pass the private tests.

6.2 How a test is scored

Some tests run more than once, to catch bugs that depend on timing. Every run must pass, and the test earns the credit of its **worst** run.

A run earns **a score of 0** when, among other things:

- Any `fetch` gets a wrong `<value>`, no reply within the time limit, or a reply on the wrong channel.
- A reply channel carries a line that answers no `fetch`.
- A `create` does not start exactly one new process, which must be a child process of the receiving node.
- A child process any `create` command does not run your `sp` or fails to open both its FIFOs.
- **SP main** does not run the consultant exactly once for each miss that reaches it, and once more for each failed run it retries.
- A node stops reading its commands, its reply channel closes while it is alive, or it writes any invalid reply.
- The process tree structure is wrong when the judge inspects it. This check is performed occasionally after all previous `fetch` commands are answered, and excludes the leftovers of a

Using an AI coding agent, such as Claude Code or Codex, to complete this homework for you counts as cheating.

`kill` (they are counted under Section 6.2.2 instead). The process tree structure is considered wrong if:

1. Any consultant run is still there without being reaped.
2. Any other process that is not a node is created under **SP main**
3. A node has been terminated unexpectedly, or its parent is not the node it was created under.

- A hard resource limit is exceeded, refer to resource limits (Section 6.2.1).
- **SP main** exits with a status other than `0` after a `kill` sent to it.
- In group 4, a `kill` command leaving anything behind.

6.2.1 Resource limits

Each test sets a soft limit and a hard limit on four axes:

1. Number of processes
2. Number of open file descriptors
3. Peak memory usage of each process
4. CPU time

Note that peak memory usage is counted at every moment of the run.

In the public tests, the soft memory limit is **64 MiB** per process (32 MiB in the test with the longest value), the soft CPU time limit is **5 seconds**, and every hard limit is **twice** its soft limit. The file of each public test lists its own limits.

- Exceeding a hard limit fails the run.
- Exceeding the soft limit on one axis deducts 50% of the test's credit, and on two or more axes deducts 75%.

6.2.2 Leftovers

Anything that a `kill` should have removed but did not, whether it is a process, an unreaped child, or a FIFO path, is a leftover. Each leftover deducts an additional **10%** of the test's credit in groups 1 to 3, and **fails the run** in group 4. Combined deductions cannot exceed the value of the test.

6.2.3 Build

The judge runs `$ make` once before any test. If the build fails or times out, **the entire assignment scores 0**.

6.3 Final score

Your score out of 100 is the sum of the credit earned by every test. Your final score is that sum normalized to 7:

$$\text{final} = \text{score} \times \frac{7}{100}$$

> **Example**
>
> Full credit in groups 1 to 3 earns $7 + 21 + 29 = 57$ points. Passing every public test of group 4, but only half of its private part, adds $12.9 + \frac{30.1}{2} = 27.95$ points. The score is then 84.95, and the final score is $84.95 \times \frac{7}{100} = 5.9465$.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

6.4 Public test

Start from the student template. On the workstation, run the public test suite from your submission root:

```shell
make test
```

This builds `sp` and runs the bundled judge against the public tests. The judge scores them alone, out of 100 with the same group points, hence passing every public test shows 100 / 100. When we grade, the public tests count for 30% of each group.

While the dashboard of the judge is on screen, the judge discards everything written by your program and its consultant runs.

> **Tip**
>
> The bundle contains only the public tests and the default consultant. Do not add judge files, test plans, or files generated by the public test to your submission. You are suggested to use `make submission` with student id filled properly to create the submission bundle.

6.5 Rules against plagiarism

> **Warning**
>
> - Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.
> - You are encouraged to discuss ideas with others and to ask AI agents questions, but you must write your own code.
> - Other forms of plagiarism are also forbidden.

If you cheat, **your score will be 0**, and we will report this accordingly.

7 Submission

Submit `<student_id>.zip` (e.g., `r15922177.zip`) to NTU COOL. GitHub Issues is for questions only, hence GitHub Classroom or pushing to a student repository is not required.

Include `sp.c`, `sp.h` and `Makefile` in your submission, along with any additional C source or header files required to build your solution. After extraction, `make` must build `sp` from all of your sources, and `make clean` must remove the files generated by the build.

> ⚠ **Zip structure**
>
> Place the files **directly** at the top level of the zip. **DO NOT** wrap them in an extra folder.
>
> ```
> r15922177.zip
> ├── sp.c
> ├── sp.h
> ├── Makefile
> └── (any other .c / .h files your solution needs)
> ```

Do not submit binaries (including `sp`), object files, judge files, test plans, files generated by the public self-test, core dumps, or any other generated files (including FIFOs).

The submission link will be provided by the course announcement on NTU COOL.

7.1 Late policy

The soft deadline is **November 5, 2026 (Thursday), 23:59**. Late submissions are accepted until the hard deadline, **November 12, 2026 (Thursday), 23:59**. All dates and times use **Taiwan time (UTC+8)**.

Your final score is multiplied by the following factor:

Submission date (2026)	Days late	Multiplier
November 6-7	1-2	0.85
November 8-10	3-5	0.70
November 11-12	6-7	0.50

Table 2: Late submission multipliers.

Submissions after November 12, 2026, 23:59 are not accepted.

We grade your **latest submission on NTU COOL**, and use its timestamp to determine the late penalty. Note that resubmitting after the soft deadline counts as late, even if you only change comments or formatting. A previous on-time submission does not exempt a later resubmission from the penalty.

Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

8 FAQ

Q1: How do I write error code?

By "write error code" we do not only mean code that tells you **where** your program stopped; we also need to know **why**. Use `perror()` (or `strerror(errno)`) so that a failed `open()` tells you whether the path was wrong, the direction was wrong, or the FIFO did not exist.

When you contact the TAs, please provide a description **and the reason for failure** (at minimum the error return value). Throwing your whole code at us is about as useful as telling us that SP stands for Systems Programming.

Q2: My `mkfifo()` returns "Operation not supported".

This is most likely because you are running on WSL and your homework directory does not support this operation. On your local machine, you can work around it by creating the FIFO under `/tmp/` to test your understanding of FIFOs, the judge may not use them accordingly.
PS: remember to delete them.

Ultimately, please run the judge and your work on the workstation. There, the judge passes FIFO paths in a directory where `mkfifo` works, and your node must create its FIFOs at exactly the paths provided in `create` arguments.

Q3: Can a node keep a copy of a value it passed on, to answer faster next time?

No. A key-value pair lives only in the node that received its `store`, and every miss must repeat the walk up the tree and, at SP main, run the consultant again. The judge counts consultant runs, and a cached answer eventually goes stale.

Q4: Can I print progress messages so I can see what is happening?

Not to STDOUT, which is a protocol channel. Write them to STDERR. The judge's dashboard hides STDERR, so run `make test > report.txt` to see it later, and remove unnecessary diagnostics before submission.

Q5: Do I have to modify the Makefile?

Yes. `make` must build your `sp` from all your sources, and `make clean` must remove the files the build generates, while the template's `Makefile` builds `sp` from `sp.c` alone. Keep its `test` target, which runs the public self-test.

Q6: My `kill` never finishes, or the judge reports a leftover process.

Usually some process still holds a write end of a pipe, so its reader never sees EOF. Close the pipe ends each side does not use right after `fork`, set `FD_CLOEXEC` on descriptors that new nodes and consultant runs must not inherit, and reap every child with `waitpid()` as soon as it exits.

Q7: Why has the TA not replied to my issue?

Please raise issues in the template repository rather than your own private homework repository.

Q8: Will the Spec be updated in the future?

Preferably no, any questions should be raised as a GitHub Issue and will be answered there. But if we deem any information necessary to be immediately specified, we will make a course announcement and update the Spec accordingly.

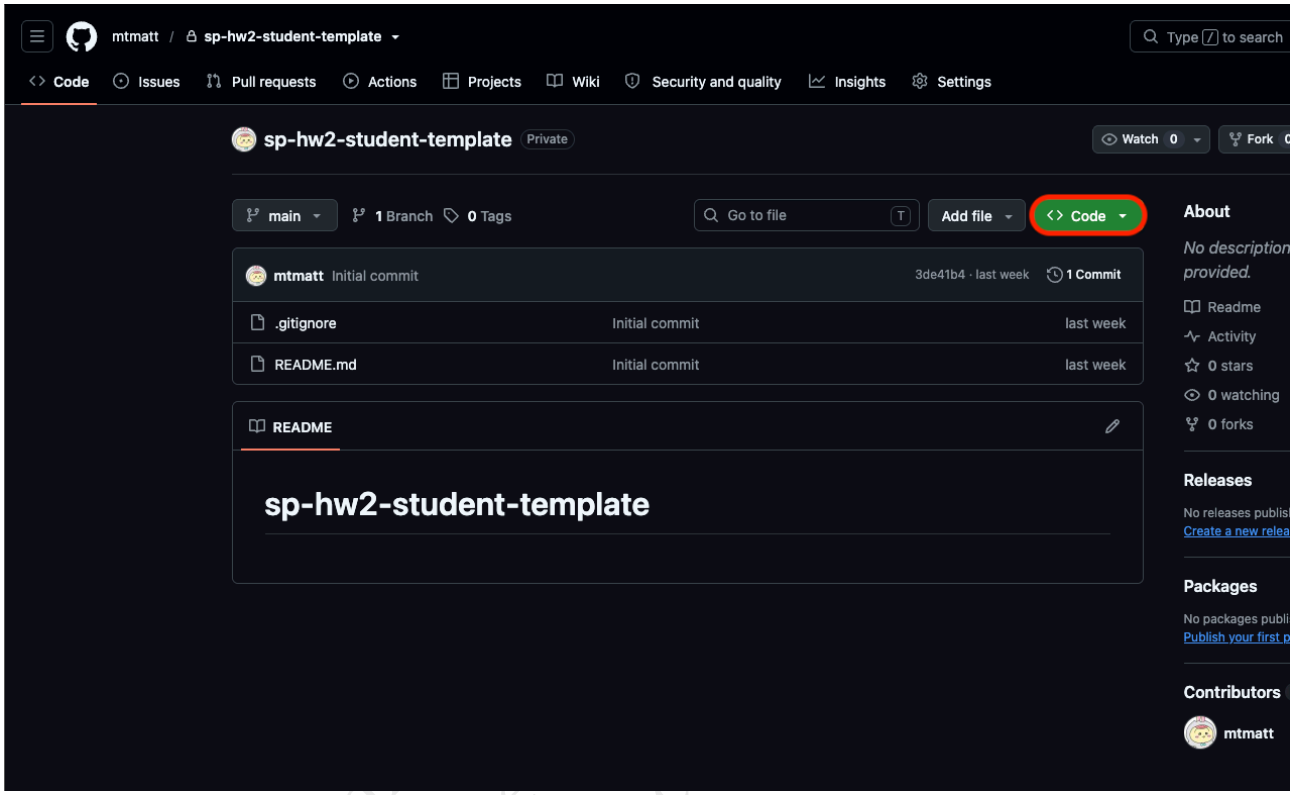
Using an AI coding agent, such as Claude Code or Codex, to complete the homework for you counts as cheating.

Appendix

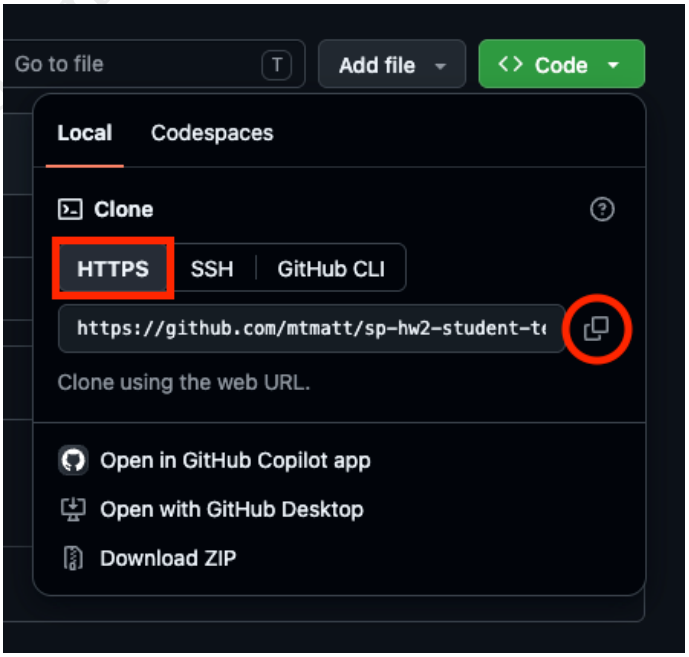
A. How to clone the template repository?

To clone the template repository, you may click the link at the beginning of this document. (or https://github.com/mtmatt/sp-hw2-student-template).

Then, click the code button.



Next, click the HTTPS, and following with the copy icon. If you want to use GitHub as well, please refer to the next section.



On your terminal, type `git clone`, paste your link and press enter:

```shell
git clone <paste-your-link-here>
```

If the clone process successfully finished, you can check it with:

```shell
> ls -l
drwxr-xr-x@    - matt 21 Sep 11:30 sp-hw2-student-template
```

You can change the name of the directory (`sp-hw2-student-template`) to whatever you want by using `mv`. For example:

```shell
mv sp-hw2-student-template sp-hw2-b12902033
```

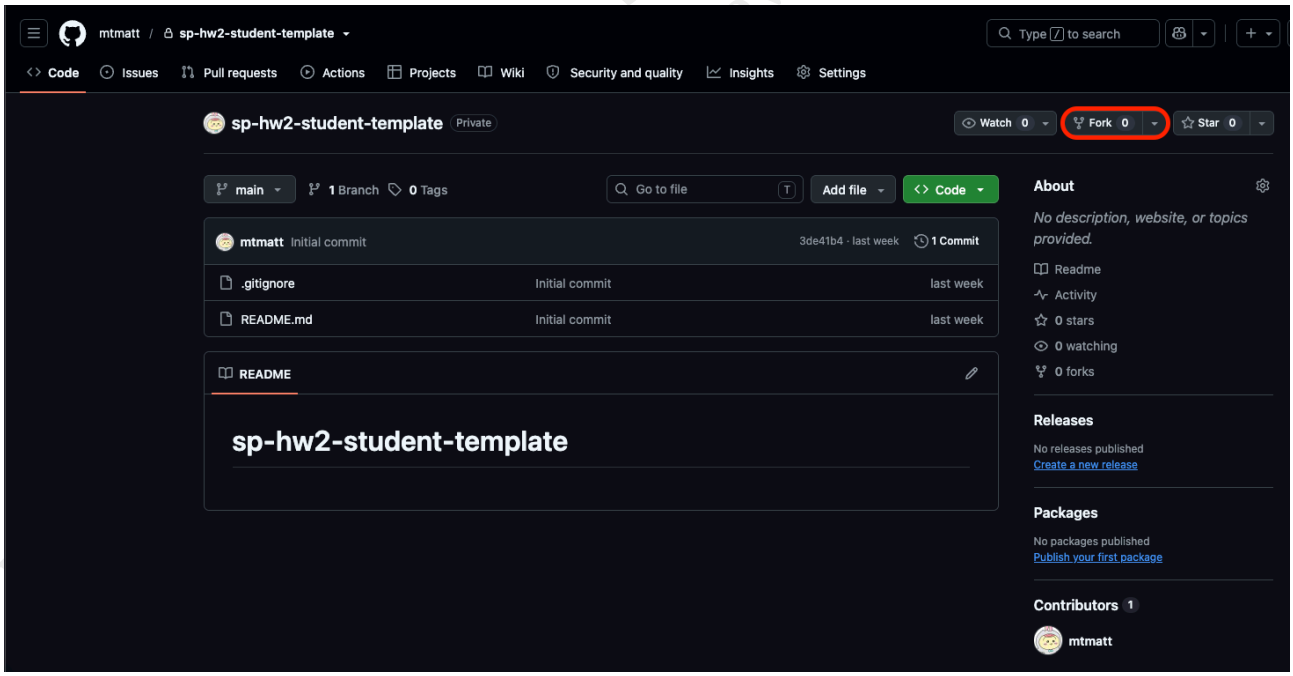
That's it. Now you have your repository set. Congratulation!!!

B. Version Control

During DSA course, you have used some `git` commands to submit your work. However, do you know there's much more possibilities to work with `git` and GitHub.

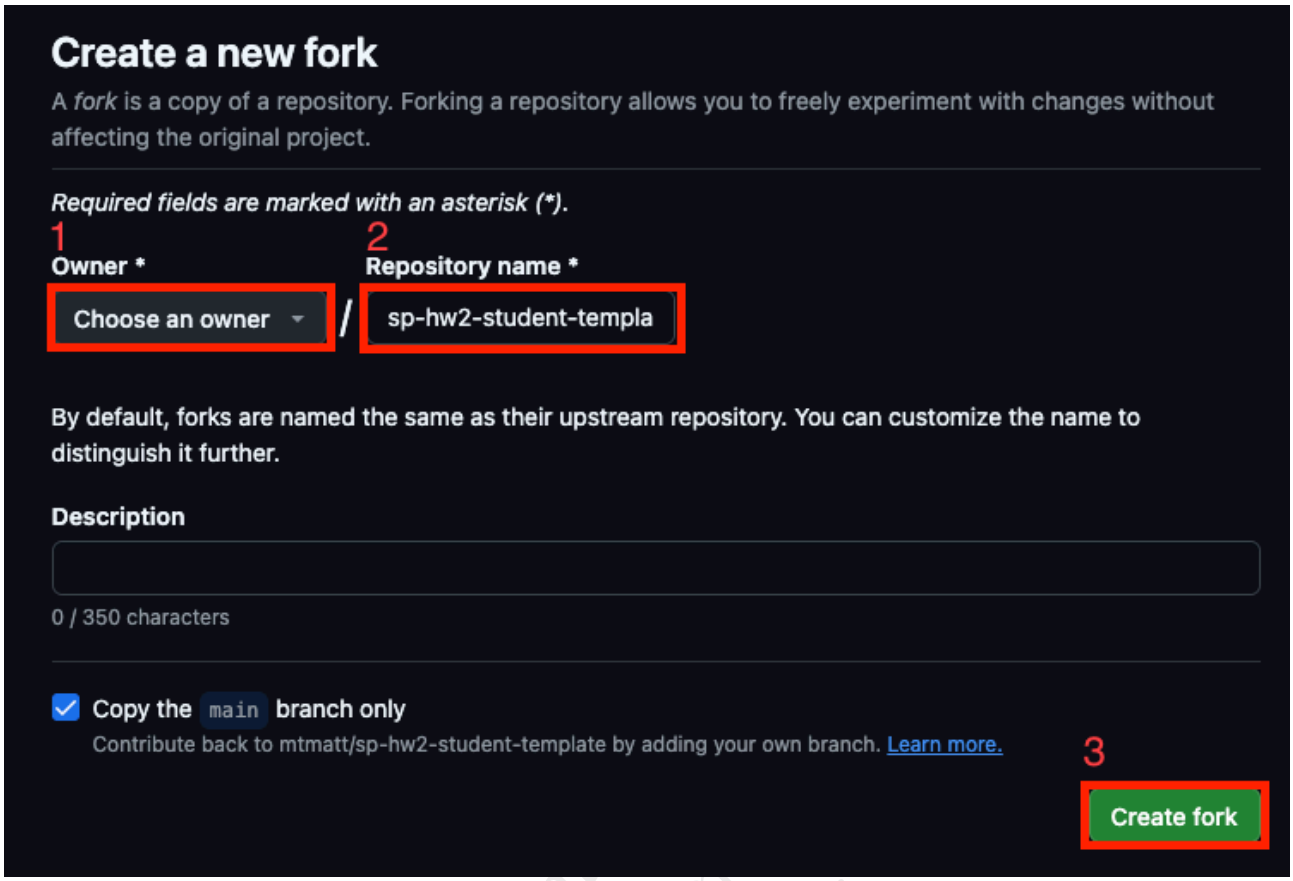
If you want to use the version control system with GitHub, we suggest that you can use **fork**.

B.1 Fork



You will go through some setting once you click the fork button.

1. Owner: You can chose yourself as the owner.
2. Repository name: You can rename the repository as you want.
3. Copy the `main` branch only: This allows you to fork faster, although this repo is quite small.



After these step, you will have your own repository.

B.2 Branch

A part from `add`, `commit`, `push`, and `pull`, there's another useful command called `switch`. If you want to create a branch, you can use:

```shell
git switch -c <branch_name>
```

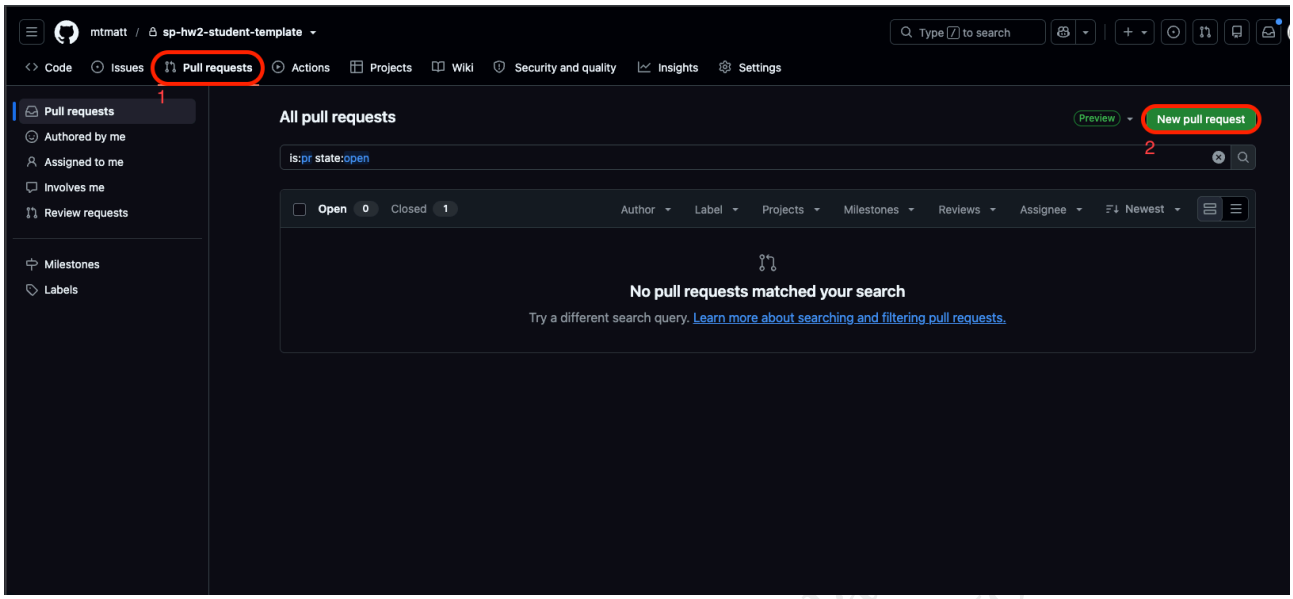
The new branch will be place on top of the current commit. Once a branch created, you can move around these branches by using:

```shell
git switch <branch_name>
```

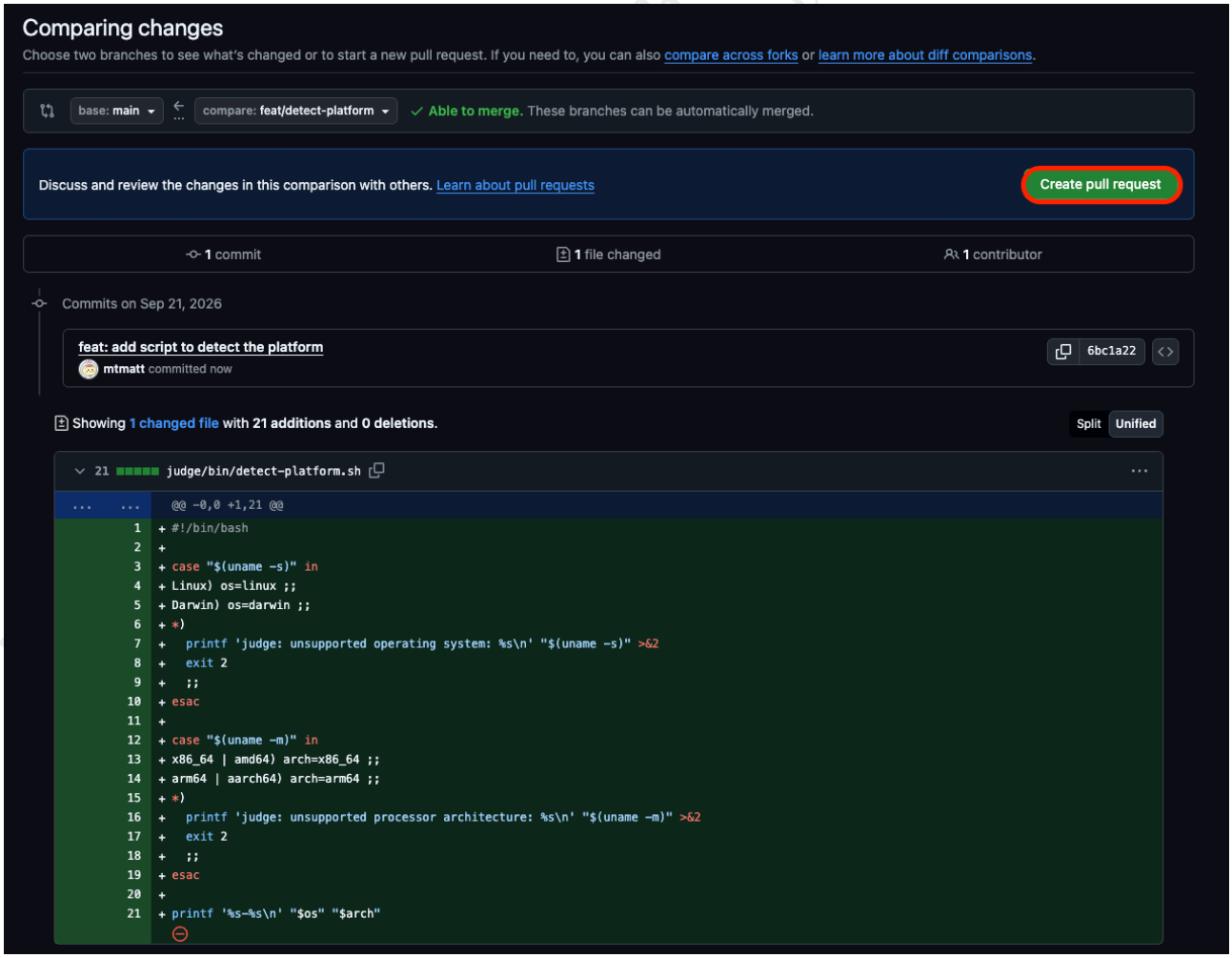
Then, when you commit some changes on a branch, you can push it to remote (in this case: GitHub).

```shell
git push -u origin <branch_name>
```

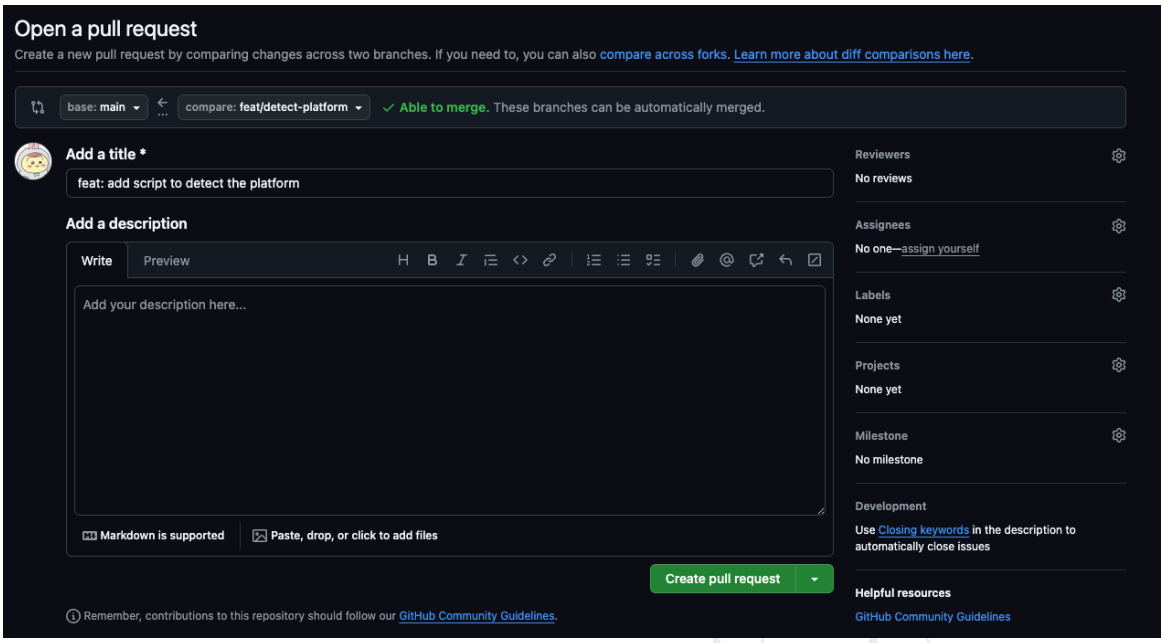
After that, you may want to create a pull request for the new branch.



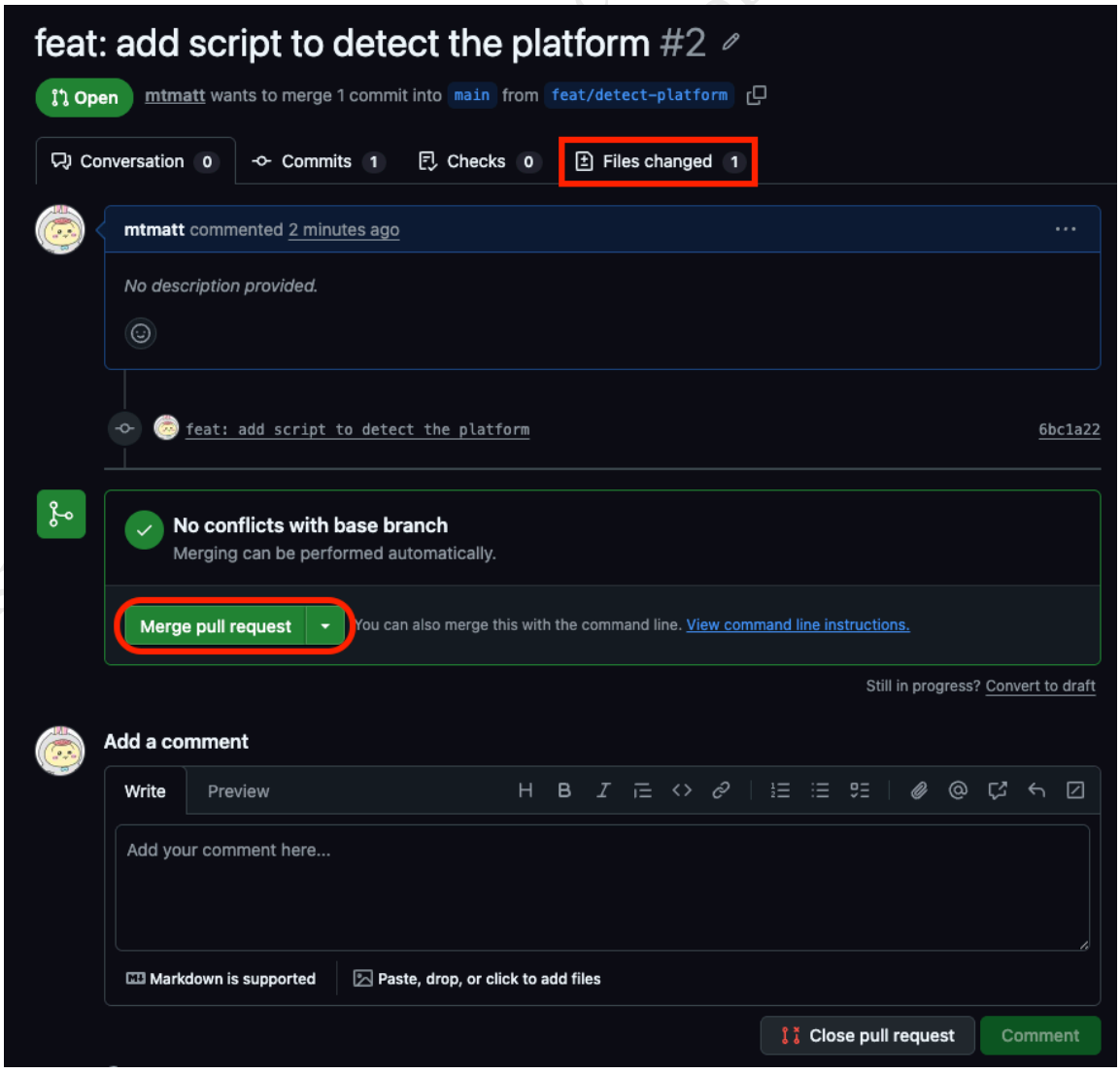
To create a new pull request, you will have to select the base (usually main), and compare (your new branch). Click “Create pull request”.



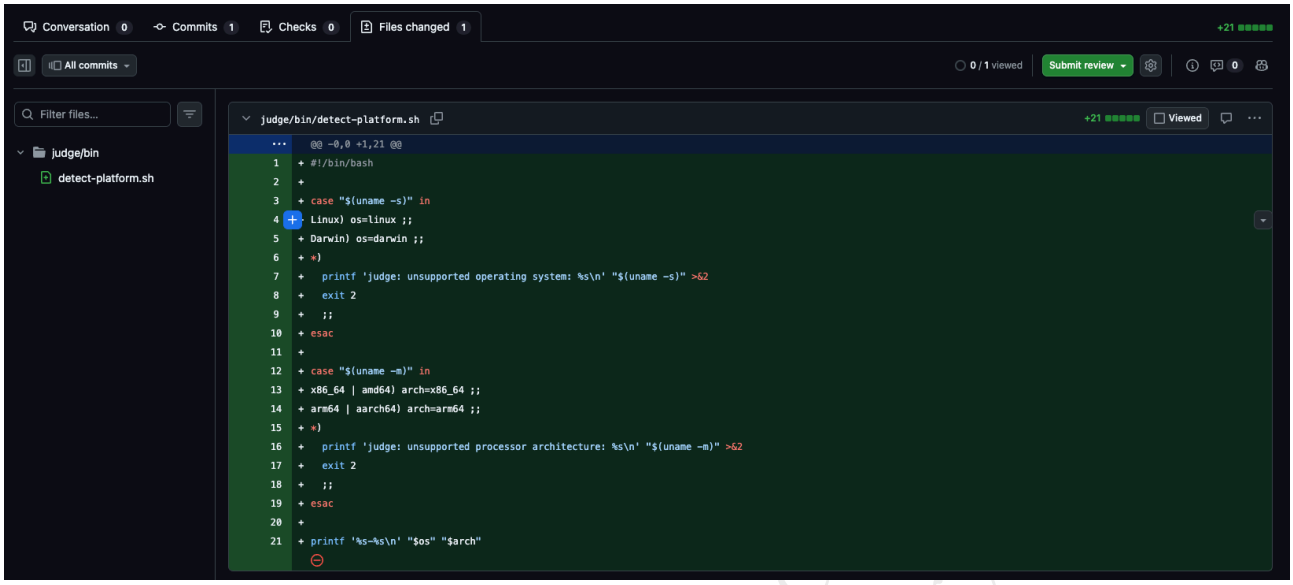
Next, you can write description if you want to let other understand what’s the purpose for this pull request. In personal assignment, this is not strictly required.



In a PR, mostly you will use these two button. The upper one helps you review your code. The lower one can merge this change into the main branch.



This is how the review page looks like.



After you merge a pull request, you may want to sync your local repository to the latest version. You can do this with:

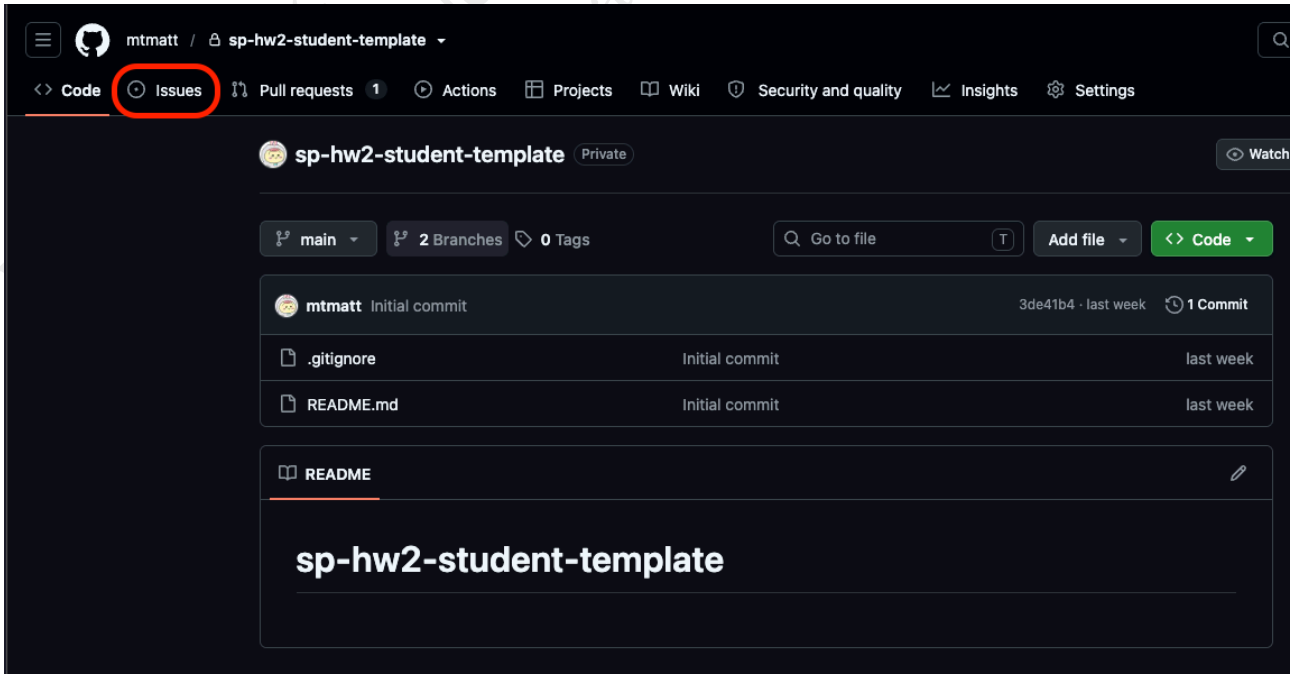
```shell
git switch main
git pull
```

Now you understand the basic to use version control properly. Congratulation!!!

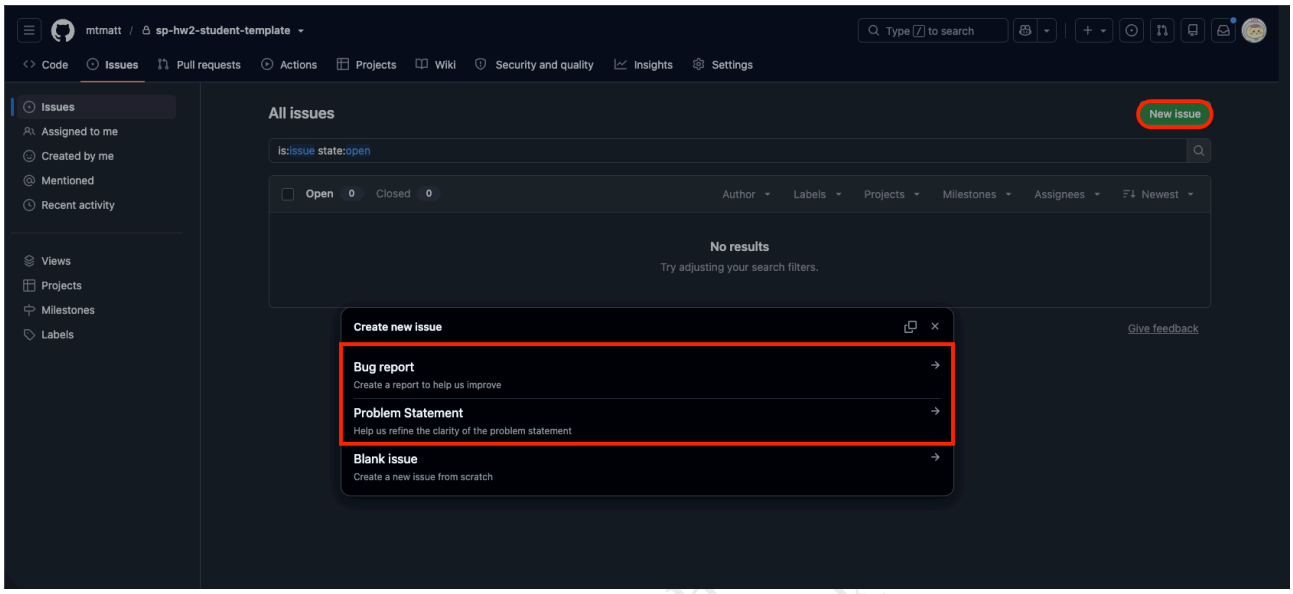
C. Issue

If you are not familiar with the GitHub Issue System, here's what you need to know.

First, you will navigate to the Issues page.



Click “New issue” button and select the template. If you find a bug from the judge, use bug report. If you encounter an issue with the problem statement, select the problem statement. You **must not** create a blank issue unless your issue is not related to these two categories.



Then you can fill the title and description of the issue. Finally, you can click “Create” to publish the issue.

